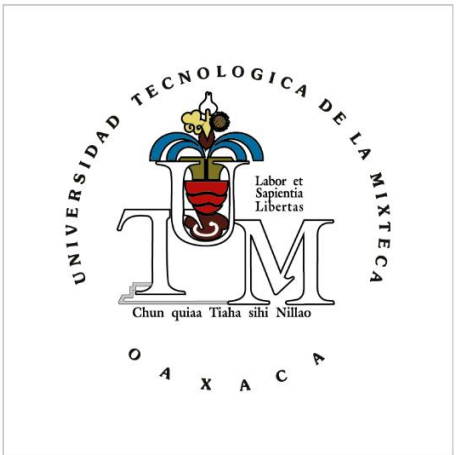
ANALIZADOR SINTÁCTICO PASCAL – C
DOCUMENTACIÓN DEL ASIGN ENTRE LOS INTEGRANTES



Equipo 6

Integrantes:

Sánchez Aguilar Luis Ángel
Álvarez Olivera Irving Damián
Ramírez Manzano Jaret
García Barela Ángel Andrés
López Osorio Haniel
Martinez Cesar Zaid
Soriano Martínez Miguel Ángel

Materia: Compiladores

Profesor: David Martínez Torres

Fecha de entrega: Viernes 9 de octubre del 2026

UNIVERSIDAD TECNOLÓGICA DE LA MIXTECA

Arquitectura de un Analizador Sintáctico de Código Fuente en Pascal

Índice

1. Introducción
2. Descripción general del sistema
3. Objetivo del programa
4. Arquitectura del sistema
    - 4.1. Entrada del código fuente
    - 4.2. Análisis sintáctico
    - 4.3. Manejo de errores
    - 4.4. Generación de resultados
5. Tecnologías utilizadas
6. Flujo de funcionamiento
7. Conclusiones
8. Tabla de asignaciones
9. Asignaciones de la semana
10. Informe de desarrollo del proyecto: PRIMEROS y SIGUIENTES
    - 10.1. Contexto y alcance del proyecto
    - 10.2. Objetivos del proyecto
    - 10.3. Fundamento teórico
    - 10.4. Arquitectura del módulo PRIMEROS y SIGUIENTES
    - 10.5. Diagrama de clases UML
    - 10.6. Descripción de los componentes
    - 10.7. Lectura y validación de gramáticas
    - 10.8. Interfaz gráfica
    - 10.9. Flujo de ejecución
    - 10.10. Ejemplo ilustrativo de cálculo
    - 10.11. Pruebas y verificación
    - 10.12. Participación del equipo en el desarrollo
    - 10.13. Conclusiones del proyecto

1. Introducción

El presente documento describe la arquitectura de un programa desarrollado en Python cuyo propósito es analizar código fuente escrito en Pascal. El sistema recibe un código fuente como entrada y procesa su estructura para determinar si cumple con las reglas sintácticas establecidas por el lenguaje.

El programa está diseñado como una herramienta de análisis que permite identificar errores en la estructura del código y proporcionar información sobre el resultado del análisis.

2. Descripción general del sistema

El sistema funciona como un analizador sintáctico de código fuente en Pascal. Su funcionamiento se basa en recibir un programa escrito en Pascal, procesarlo y verificar que sus diferentes elementos se encuentren organizados de acuerdo con la sintaxis del lenguaje.

El software está desarrollado en Python y está compuesto por diferentes etapas de procesamiento. Cada etapa recibe la información generada por la anterior y realiza una tarea específica hasta obtener el resultado final del análisis.

De manera general, el procesamiento sigue el siguiente esquema:

Código fuente → Análisis léxico → Análisis sintáctico → Detección de errores → Resultado

3. Objetivo del programa

El objetivo principal del programa es analizar la estructura sintáctica de código fuente escrito en Pascal, permitiendo detectar construcciones que no cumplen con las reglas definidas para el lenguaje.

Entre sus principales funciones se encuentran:

- Recibir código fuente escrito en Pascal.
- Identificar los elementos que componen el código.
- Comprobar la estructura y el orden de dichos elementos.
- Detectar errores sintácticos.
- Indicar información relacionada con los errores encontrados.
- Mostrar el resultado del análisis al usuario.

4. Arquitectura del sistema

La arquitectura del programa se organiza en varias etapas que trabajan de manera secuencial. Esta división permite separar las responsabilidades de cada componente y facilita el mantenimiento y modificación del software.

4.1. Entrada del código fuente

Esta etapa se encarga de recibir el código fuente que será analizado.

El código puede ser proporcionado mediante un archivo o mediante el mecanismo de entrada implementado por el programa. Una vez recibido, el contenido se envía a las siguientes etapas para su procesamiento.

Entrada: código fuente en Pascal.
Salida: código preparado para su procesamiento.

4.2. Análisis sintáctico

El análisis sintáctico es la etapa principal del sistema. Su función es comprobar que la secuencia de tokens cumple con las reglas gramaticales establecidas para Pascal.

El analizador verifica aspectos como la correcta estructura de las declaraciones, instrucciones, expresiones, bloques de código y demás construcciones soportadas por el programa.

Para realizar esta comprobación, el analizador procesa los tokens siguiendo las reglas de la gramática definida para el lenguaje.

Entrada: secuencia de tokens.
Salida: estructura sintácticamente válida o información sobre los errores encontrados.

4.3. Manejo de errores

El sistema cuenta con un mecanismo para detectar y reportar errores durante el proceso de análisis.

Cuando se encuentra una estructura que no cumple con la sintaxis esperada, el programa genera información que permite identificar el problema. Dependiendo de la implementación, esta información puede incluir el tipo de error y su ubicación dentro del código fuente.

El manejo de errores permite que el usuario conozca qué parte del programa debe revisar o corregir.

4.4. Generación de resultados

Una vez finalizado el análisis, el sistema presenta el resultado al usuario.

Si el código cumple con las reglas sintácticas implementadas, se indica que el análisis fue satisfactorio. En caso contrario, se muestran los errores detectados durante el procesamiento.

Resultado: código válido o lista de errores sintácticos.

5. Tecnologías utilizadas

Para el desarrollo del sistema se utilizan principalmente las siguientes tecnologías:

Tecnología	Función
Python	Lenguaje utilizado para desarrollar el analizador.
Pascal	Lenguaje del código fuente que será analizado.
Gramática sintáctica	Define las reglas que debe cumplir el código Pascal.

Python permite implementar los diferentes componentes del analizador y controlar el flujo de procesamiento del código fuente.

6. Flujo de funcionamiento

El funcionamiento general del programa puede representarse mediante las siguientes etapas:

1. El usuario proporciona un código fuente escrito en Pascal.
2. El programa recibe y prepara el código para su análisis.
3. El analizador léxico identifica y clasifica los tokens.
4. Los tokens son enviados al analizador sintáctico.
5. El analizador sintáctico compara la estructura recibida con las reglas de la gramática.
6. Si encuentra una estructura incorrecta, registra el error correspondiente.
7. Al finalizar el procesamiento, el programa muestra el resultado del análisis.

De esta manera, el sistema transforma progresivamente el código fuente desde su representación textual hasta un resultado que indica si su estructura sintáctica es correcta.

7. Conclusiones

El analizador sintáctico desarrollado en Python permite procesar código fuente escrito en Pascal y comprobar que su estructura cumpla con las reglas sintácticas establecidas.

La arquitectura basada en etapas de entrada, análisis léxico, análisis sintáctico, manejo de errores y generación de resultados permite organizar el funcionamiento del programa de manera clara y facilita su mantenimiento.

El sistema constituye una herramienta útil para identificar errores sintácticos y comprender el proceso de análisis de un lenguaje de programación como Pascal.

8. Tabla de asignaciones

NOMBRE	ACTIVIDAD	DESCRIPCIÓN	ENTREGABLE
Luis Ángel Sánchez Aguilar	Coordinación e integración	Organizar las actividades, integrar la lectura de gramáticas, los algoritmos y la interfaz. Verificar el funcionamiento completo.	Plan de trabajo, programa integrado y versión final lista para entregar.

NOMBRE	ACTIVIDAD	DESCRIPCIÓN	ENTREGABLE
Ángel Andrés García Barela	Lectura y validación de gramáticas	Leer desde un archivo los terminales, no terminales, símbolo inicial y producciones. Reconocer símbolos como id, V' y ε; detectar datos faltantes o inconsistentes.	Módulo de lectura y validación, formato de archivo documentado y archivos de las gramáticas de la asignación.
Haniel López Osorio	Algoritmo de PRIMEROS	Implementar el cálculo de PRIMEROS para todos los no terminales, considerando producciones vacías, cadenas de símbolos y recursión. Repetir el cálculo hasta que los conjuntos dejen de cambiar.	Módulo de PRIMEROS y ejemplos verificados de sus resultados.
Irving Damián Álvarez Olivera	Algoritmo de SIGUIENTES	Implementar el cálculo de SIGUIENTES utilizando PRIMEROS. Incorporar $ al símbolo inicial y propagar los conjuntos hasta que dejen de cambiar, sin incluir ε en SIGUIENTES.	Módulo de SIGUIENTES y ejemplos verificados de sus resultados.
Jaret Ramírez Manzano	Interfaz gráfica	Diseñar la ventana para cargar el archivo, visualizar la gramática, ejecutar el cálculo y mostrar PRIMEROS, SIGUIENTES y mensajes de error de forma legible.	Interfaz gráfica funcional y capturas para el informe.
Miguel Ángel Soriano Martínez	Pruebas y verificación	Calcular manualmente los resultados esperados y compararlos con el programa. Priorizar las gramáticas que incluyen E → E+T, D → DD y S → TV;, además de los demás ejemplos. Registrar fallas y comprobar sus correcciones.	Tabla de pruebas con resultados esperados y obtenidos, evidencias y registro de errores corregidos.
Cesar Zaid Martinez	Arquitectura, UML e informe	Proponer la organización de clases junto con los responsables de los módulos. Elaborar y actualizar el diagrama de clases; reunir las aportaciones del equipo y preparar el informe con índice y planificación.	Diagrama de clases, descripción de la arquitectura e informe final con índice y evidencias.

9. Asignaciones de la semana

Día	Objetivo principal	Tareas asignadas del día	Responsables
Lunes 28	Organización y definición de gramáticas	Dividir las asignaciones del proyecto según corresponde y analizar sus requerimientos. Desarrollar las diferentes gramáticas para el desarrollo posterior de los algoritmos. Definir los símbolos o tokens terminales y no terminales.	García Barela Ángel Andrés Sánchez Aguilar Luis Ángel
Martes 29	Desarrollo de algoritmos	Implementar el algoritmo de PRIMEROS para todos los símbolos terminales de las gramáticas. Implementar el algoritmo de SIGUIENTES para todos los símbolos terminales.	Álvarez Olivera Irving Damián López Osorio Haniel
Miércoles 30	Implementación de la interfaz	Comenzar con el diseño técnico de la interfaz gráfica donde se ejecutarán los algoritmos.	Ramírez Manzano Jaret Martinez Cesar Zaid
Jueves 1 de octubre	Pruebas y correcciones	Calcular los resultados de forma manual para corroborar que los resultados del programa sean correctos. Registrar posibles fallas y trabajar en su debida corrección.	Soriano Martínez Miguel Ángel Ramírez Manzano Jaret
Viernes 2	Entrega final	Proponer la organización de las clases para crear su diagrama. Terminar de registrar las asignaciones de cada integrante y organizar el respectivo documento al respecto. Reunir todas las aportaciones del equipo.	Soriano Martínez Miguel Ángel Martinez Cesar Zaid

10. Informe de desarrollo del proyecto: PRIMEROS y SIGUIENTES

Este apartado documenta el desarrollo del módulo de cálculo de los conjuntos PRIMEROS y SIGUIENTES, realizado por el Equipo 6 como parte del analizador sintáctico para Pascal. Se describen su propósito, su fundamento teórico, su arquitectura (con el diagrama de clases UML), la interfaz gráfica, el flujo de ejecución, la estrategia de pruebas y la participación de cada integrante.

10.1. Contexto y alcance del proyecto

El análisis sintáctico descrito en las secciones anteriores se basa en una gramática que define las reglas del lenguaje. Para que un analizador predictivo pueda decidir qué producción aplicar y detectar errores con precisión, es necesario conocer previamente dos conjuntos asociados a cada no terminal de la gramática: PRIMEROS y SIGUIENTES. Estos conjuntos son la base para construir la tabla de análisis sintáctico.

El proyecto PRIMEROS y SIGUIENTES implementa estos cálculos como una aplicación en Python con interfaz gráfica. La aplicación recibe una gramática desde un archivo, la valida, calcula ambos conjuntos para todos los no terminales y presenta los resultados de forma legible, con la posibilidad de exportarlos. Se integra como una pestaña (FirstFollowTab) dentro de la ventana principal (MainWindow), junto con las pestañas de los analizadores léxicos del equipo.

Esquema general del módulo:

Archivo de gramática → Lectura y validación → PRIMEROS → SIGUIENTES → Visualización / Exportación

10.2. Objetivos del proyecto

- Leer desde un archivo los terminales, no terminales, el símbolo inicial y las producciones de una gramática.
- Reconocer símbolos como id, V’ y ε, y detectar datos faltantes o inconsistentes.
- Calcular PRIMEROS para todos los no terminales, considerando producciones vacías, cadenas de símbolos y recursión.
- Calcular SIGUIENTES a partir de PRIMEROS, incorporando $ al símbolo inicial y sin incluir ε.
- Mostrar PRIMEROS, SIGUIENTES y los mensajes de error en una interfaz gráfica clara.
- Verificar los resultados contra cálculos manuales, priorizando las gramáticas E → E+T, D → DD y S → TV;.
- Documentar la arquitectura mediante un diagrama de clases UML y un informe final.

10.3. Fundamento teórico

Conjunto PRIMEROS

PRIMEROS(X) es el conjunto de terminales con los que puede comenzar alguna cadena derivada de X. Si X puede derivar la cadena vacía, ε también pertenece a PRIMEROS(X). Las reglas de cálculo son:

1. Si X es un terminal, PRIMEROS(X) = {X}.
2. Si X → ε es una producción, se agrega ε a PRIMEROS(X).
3. Si X → Y1 Y2 … Yn, se agrega a PRIMEROS(X) todo símbolo de PRIMEROS(Y1) distinto de ε. Si Y1 deriva ε, se continúa con Y2, y así sucesivamente. Si todos los Yi derivan ε, se agrega ε a PRIMEROS(X).
4. El proceso se repite hasta que ningún conjunto cambie (punto fijo), lo cual resuelve la recursión por la izquierda y las dependencias entre no terminales.

Conjunto SIGUIENTES

SIGUIENTES(A) es el conjunto de terminales que pueden aparecer inmediatamente a la derecha de A en alguna forma sentencial, incluyendo $ (fin de la entrada). Las reglas de cálculo son:

1. Se agrega $ a SIGUIENTES del símbolo inicial.
2. Para cada producción A → αBβ, se agrega a SIGUIENTES(B) todo elemento de PRIMEROS(β) distinto de ε.
3. Si β es vacío o PRIMEROS(β) contiene ε (A → αB o A → αBβ con β ⇒* ε), se agrega SIGUIENTES(A) a SIGUIENTES(B).
4. Se repite hasta que los conjuntos dejen de cambiar. ε nunca forma parte de SIGUIENTES.

10.4. Arquitectura del módulo PRIMEROS y SIGUIENTES

El módulo sigue una organización por capas, con cuatro paquetes con responsabilidades bien separadas. Esta división facilita que cada integrante trabaje en su parte y que la integración final se realice a través de interfaces claras.

Paquete	Capa	Responsabilidad
PRESENTATION	Interfaz gráfica	Ventana principal, pestaña de PRIMEROS y SIGUIENTES, tabla de resultados y ejecución del cálculo en un hilo independiente.
LOGIC · first_follow	Lógica de negocio	Algoritmos de PRIMEROS y SIGUIENTES, módulo de integración entre ambos y manejo de errores del cálculo.
PERSISTENCE	Persistencia	Lectura y validación de gramáticas desde archivo, y lectura/escritura de archivos de texto y TSV.
GRAMMAR_MODELS	Modelo de datos	Clases que representan la gramática: Grammar, Production y Symbol.

Las dependencias entre paquetes van de arriba hacia abajo: la capa de presentación usa la lógica, la persistencia y el modelo; la lógica opera sobre el modelo; y la persistencia crea instancias del modelo. El modelo no depende de ningún otro paquete.

10.5. Diagrama de clases UML

A continuación se presenta la vista general del diagrama de clases, que muestra los paquetes, las clases y módulos principales, y sus relaciones (composición, asociación y dependencias de uso, creación y lanzamiento de excepciones).

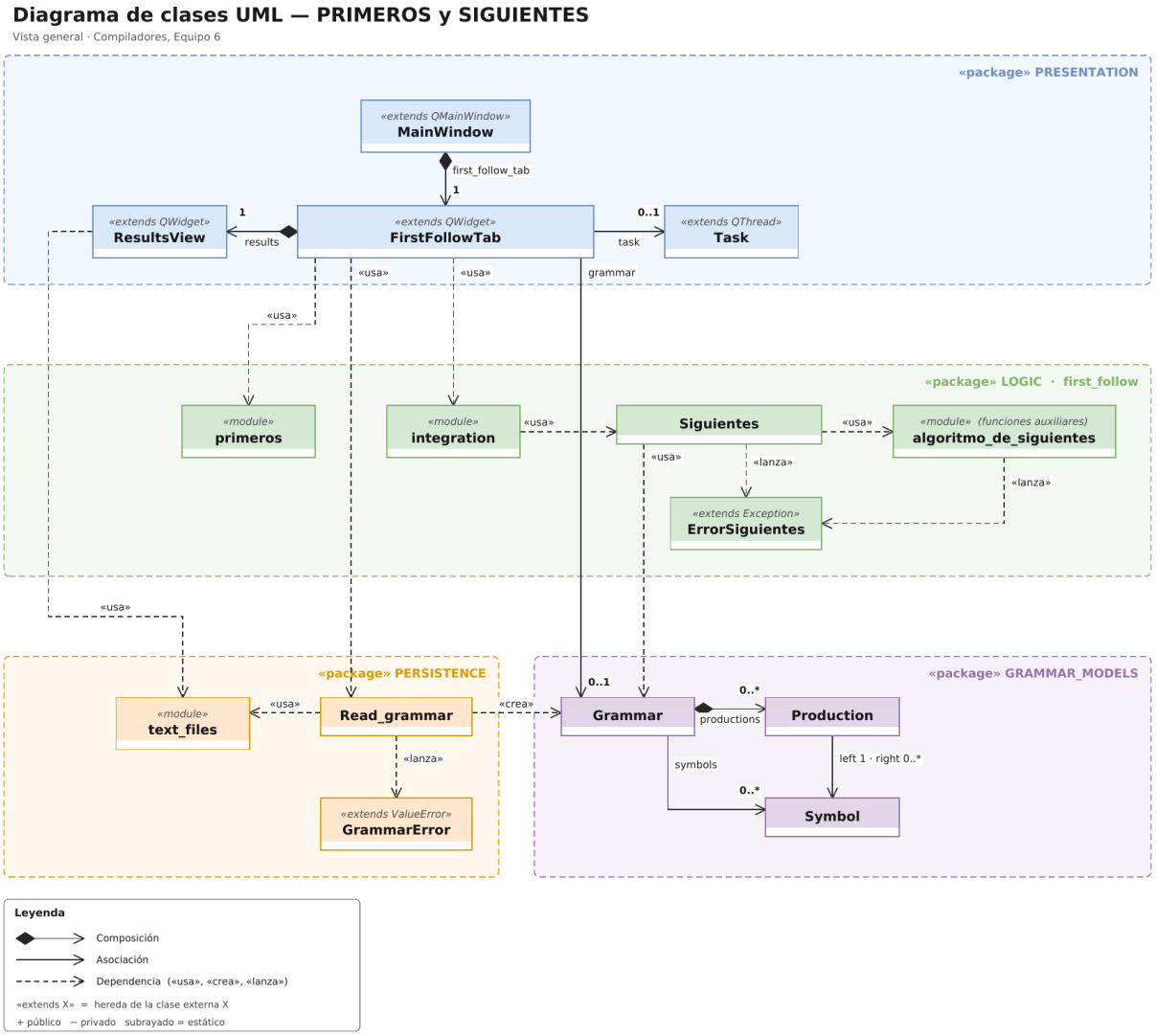


Figura 1. Diagrama de clases UML — vista general.

La siguiente figura muestra la vista de detalle, con los atributos, métodos y todas las relaciones de cada clase. En la notación, + indica visibilidad pública, − privada, y el subrayado indica miembros estáticos.

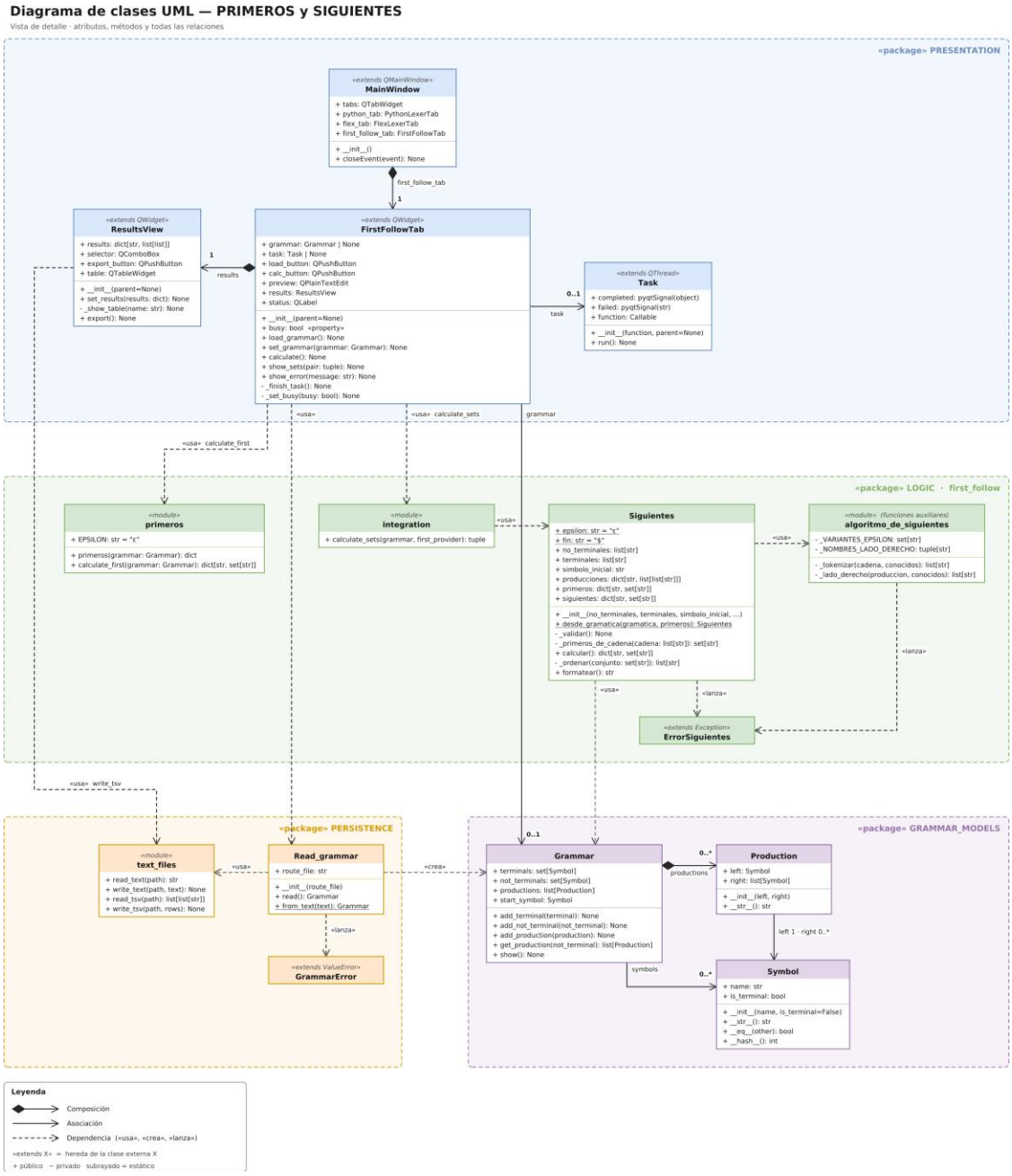


Figura 2. Diagrama de clases UML — vista de detalle.

10.6. Descripción de los componentes

Paquete PRESENTATION

Componente	Descripción
MainWindow	Hereda de QMainWindow. Contiene un QTabWidget con las pestañas del proyecto (python_tab, flex_tab y first_follow_tab). Su método closeEvent controla el cierre de la aplicación.
FirstFollowTab	Hereda de QWidget. Es la pestaña de PRIMEROS y SIGUIENTES. Mantiene la gramática cargada (grammar), el hilo de cálculo (task), los botones de cargar y calcular, una vista previa de la gramática (preview), la tabla de resultados (results) y una etiqueta de estado (status). Métodos principales: load_grammar, set_grammar, calculate, show_sets y show_error; internamente usa _finish_task y _set_busy para controlar el estado de ocupado.
ResultsView	Hereda de QWidget. Muestra los resultados en una tabla (QTableWidget) con un selector (QComboBox) para elegir qué conjunto visualizar. Ofrece set_results para cargar los datos y export para guardarlos mediante el módulo text_files.
Task	Hereda de QThread. Ejecuta una función en segundo plano y comunica el resultado mediante las señales completed (éxito) y failed (error con mensaje), evitando que la interfaz se bloquee durante el cálculo.

Paquete LOGIC · first_follow

Componente	Descripción
primeros (módulo)	Implementa el cálculo de PRIMEROS. La función calculate_first(grammar) devuelve un diccionario que asocia cada no terminal con su conjunto de PRIMEROS; primeros(grammar) es el punto de entrada del módulo. Define la constante EPSILON = "ε".
Siguientes (clase)	Implementa el cálculo de SIGUIENTES. Almacena los no terminales, terminales, símbolo inicial, producciones y los conjuntos primeros y siguientes. El método estático desde_gramatica construye una instancia a partir de un objeto Grammar y los PRIMEROS; calcular ejecuta la propagación hasta el punto fijo; _primeros_de_cadena obtiene los PRIMEROS de una cadena de símbolos; _validar verifica la consistencia de los datos; _ordenar y formatear presentan el resultado de forma ordenada. Usa ε y $ como constantes de clase.
algoritmo_de_siguientes (módulo)	Funciones auxiliares de Siguientes: _tokenizar y _lado_derecho interpretan el lado derecho de las producciones a partir de los símbolos conocidos, y se reconocen distintas variantes de escritura de ε (_VARIANTES_EPSILON).
integration (módulo)	Función calculate_sets(grammar, first_provider) que coordina ambos cálculos: obtiene PRIMEROS mediante el proveedor indicado y luego calcula SIGUIENTES, devolviendo ambos resultados como una tupla. Es el punto de conexión entre la interfaz y los algoritmos.
ErrorSiguientes	Excepción propia (hereda de Exception) lanzada cuando los datos para calcular SIGUIENTES son inválidos o inconsistentes.

Paquete PERSISTENCE

Componente	Descripción
Read_grammar	Lee una gramática desde el archivo indicado en route_file. El método read() devuelve un objeto Grammar, y el método estático from_text(text) construye la gramática a partir de una cadena de texto. Lanza GrammarError ante datos faltantes o inconsistentes.
GrammarError	Excepción propia (hereda de ValueError) para errores en la definición de la gramática.
text_files (módulo)	Funciones de entrada/salida: read_text y write_text para texto plano, y read_tsv y write_tsv para tablas separadas por tabuladores (usadas para exportar resultados).

Paquete GRAMMAR_MODELS

Componente	Descripción
Grammar	Representa la gramática completa: conjuntos de terminales y no terminales, lista de producciones y símbolo inicial. Ofrece add_terminal, add_not_terminal, add_production, get_production(not_terminal) para obtener las producciones de un no terminal, y show para mostrarla.
Production	Una producción de la gramática: un símbolo del lado izquierdo (left) y una lista de símbolos del lado derecho (right). Una gramática se compone de cero o más producciones.
Symbol	Símbolo de la gramática, con su nombre (name) y un indicador is_terminal. Implementa __eq__ y __hash__ para poder almacenarse en conjuntos y compararse por valor.

10.7. Lectura y validación de gramáticas

La lectura de gramáticas está a cargo de la clase Read_grammar. Cada gramática se define a partir de sus terminales, no terminales, símbolo inicial y producciones, y se construye como un objeto Grammar con sus Production y Symbol correspondientes.

Durante la lectura se reconocen símbolos especiales como id, V' (no terminales con apóstrofo) y ε (cadena vacía), y se validan los datos para detectar información faltante o inconsistente. Cuando se encuentra un problema, se lanza una excepción GrammarError, cuyo mensaje es mostrado al usuario en la interfaz mediante show_error.

10.8. Interfaz gráfica

La interfaz fue desarrollada con PyQt y se integra como una pestaña dentro de MainWindow. Permite al usuario:

- Cargar un archivo de gramática con el botón correspondiente (load_grammar).
- Visualizar la gramática cargada en un área de vista previa.
- Ejecutar el cálculo con el botón de calcular (calculate).

- Consultar los resultados en una tabla y alternar entre PRIMEROS y SIGUIENTES mediante un selector.
- Exportar los resultados a un archivo TSV.
- Ver mensajes de error legibles y el estado de la operación en la etiqueta de estado.

El cálculo se ejecuta en un hilo independiente (Task, basado en QThread). Mientras el hilo trabaja, la pestaña se marca como ocupada (_set_busy) para evitar acciones simultáneas; al terminar, la señal completed entrega los conjuntos calculados y la señal failed informa cualquier error.

10.9. Flujo de ejecución

1. El usuario carga un archivo de gramática desde FirstFollowTab (load_grammar).
2. Read_grammar lee el archivo y construye un objeto Grammar; si hay errores lanza GrammarError y se muestra el mensaje.
3. FirstFollowTab guarda la gramática (set_grammar) y muestra su vista previa.
4. Al presionar calcular, se crea un Task que ejecuta calculate_sets en segundo plano.
5. calculate_sets obtiene los PRIMEROS (módulo primeros) y con ellos calcula los SIGUIENTES (clase Siguientes).
6. Si el cálculo es correcto, se emite completed y show_sets entrega los resultados a ResultsView; en caso contrario se emite failed y se muestra el error.
7. El usuario consulta los conjuntos en la tabla y, si lo desea, los exporta a un archivo TSV mediante write_tsv.

10.10. Ejemplo ilustrativo de cálculo

Como ejemplo ilustrativo del funcionamiento de los algoritmos se utiliza la gramática clásica de expresiones aritméticas sin recursión por la izquierda, con E como símbolo inicial:

```
E  → T E'
E' → + T E' | ε
T  → F T'
T' → * F T' | ε
F  → ( E ) | id
```

Aplicando las reglas de la sección 10.3 se obtienen los siguientes conjuntos:

No terminal	PRIMEROS	SIGUIENTES
E	{ (, id }	{ $,) }
E'	{ +, ε }	{ $,) }
T	{ (, id }	{ +, $,) }
T'	{ *, ε }	{ +, $,) }
F	{ (, id }	{ *, +, $,) }

Este ejemplo muestra los tres comportamientos que el programa debe manejar: producciones vacías (E' → ε y T' → ε), propagación de SIGUIENTES cuando el resto de una producción deriva ε, y la presencia de $ en el símbolo inicial, sin que ε aparezca nunca en un conjunto SIGUIENTES.

10.11. Pruebas y verificación

La verificación se realizó comparando los resultados calculados manualmente con los obtenidos por el programa. El procedimiento seguido fue:

1. Calcular a mano PRIMEROS y SIGUIENTES de cada gramática de la asignación aplicando las reglas de la sección 10.3.
2. Ejecutar el programa con la misma gramática y comparar los conjuntos obtenidos con los esperados.
3. Registrar cualquier diferencia como una falla, corregirla y repetir la comprobación.

Se dio prioridad a las gramáticas que incluyen recursión por la izquierda (E → E+T), recursión que genera ambigüedad entre símbolos adyacentes (D → DD) y producciones con símbolos terminales finales (S → TV;), por ser las que más exigen el cálculo de punto fijo y la propagación entre conjuntos. También se verificó que:

- ε no aparezca en ningún conjunto SIGUIENTES.
- $ pertenezca a SIGUIENTES del símbolo inicial.
- Las gramáticas con datos faltantes o inconsistentes generen un mensaje de error claro en lugar de un fallo del programa.

Los resultados esperados y obtenidos, las evidencias y el registro de errores corregidos se conservan en la tabla de pruebas entregada por el responsable de esta actividad.

10.12. Participación del equipo en el desarrollo

La siguiente tabla relaciona la responsabilidad de cada integrante con los componentes del diagrama de clases:

Integrante	Responsabilidad	Componentes relacionados
Luis Ángel Sánchez Aguilar	Coordinación e integración	integration, MainWindow y la integración general de lectura, algoritmos e interfaz.
Ángel Andrés García Barela	Lectura y validación de gramáticas	Read_grammar, GrammarError, text_files y los modelos Grammar, Production y Symbol.
Haniel López Osorio	Algoritmo de PRIMEROS	Módulo primeros (primeros, calculate_first).
Irving Damián Álvarez Olivera	Algoritmo de SIGUIENTES	Clase Siguientes, algoritmo_de_siguientes y ErrorSiguientes.
Jaret Ramírez Manzano	Interfaz gráfica	FirstFollowTab, ResultsView y Task.

Integrante	Responsabilidad	Componentes relacionados
Miguel Ángel Soriano Martínez	Pruebas y verificación	Validación de todos los módulos con cálculos manuales y registro de fallas.
Cesar Zaid Martinez	Arquitectura, UML e informe	Diagrama de clases UML y este informe.

10.13. Conclusiones del proyecto

El proyecto permitió implementar y validar el cálculo de los conjuntos PRIMEROS y SIGUIENTES, que son un requisito para construir el analizador sintáctico de Pascal. La arquitectura en cuatro paquetes (presentación, lógica, persistencia y modelos de gramática) separó las responsabilidades, permitió el trabajo en paralelo del equipo y facilitó la integración final.

El uso de un hilo independiente para el cálculo mantiene la interfaz fluida, la validación de gramáticas evita resultados incorrectos por datos mal definidos y la exportación de resultados facilita su uso posterior en la construcción de la tabla de análisis. El diagrama de clases UML documenta de forma clara la estructura del sistema y servirá como guía para futuras ampliaciones del proyecto.